Cloud-containerization-and-DevOps

Experiment 12: Kubernetes

Introduction

Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run your app
Large ecosystem	Many tools and plugins available
Cloud-native support	Works on AWS, Google Cloud, Azure, etc.

Core Kubernetes Concepts (Simple Explanation)

Docker Concept	Kubernetes Equivalent	What it means
Container	**Pod**	A pod is a group of one or more containers. Smallest unit in K8s.
Compose service	**Deployment**	Describes how your app should run (e.g., 2 copies, which image to use)
Load balancing	**Service**	Exposes your app to the outside world or other pods
Scaling	**ReplicaSet**	Ensures a certain number of pod copies are always running

Hands-On

Step-1:- Create a `wordpress-deployment.yaml`

```
nano wordpress-deployment.yaml
```

Paste This:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ cat wordpress-deployment.ya
ml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress

spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress

  template:
    metadata:
      labels:
        app: wordpress

    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

Step-2:- Apply Deployment

```
kubectl apply -f wordpress-deployment.yaml
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ kubectl apply -f wordpress-
deployment.yaml
deployment.apps/wordpress created
```

Step-3:- Create `wordpress-service.yaml`

```
nano wordpress-service.yaml
```

Paste this:-

```yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ cat wordpress-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Step-4:- Deploy Service

```
kubectl apply -f wordpress-service.yaml
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ kubectl apply -f wordpress-
service.yaml
service/wordpress-service created
```

Step-5:- Get pods

```
kubectl get pods
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ kubectl get pods
NAME                          READY   STATUS              RESTARTS   AGE
wordpress-6698dd7d66-6t6cp    0/1     ContainerCreating   0          65s
wordpress-6698dd7d66-x5bh9    0/1     ContainerCreating   0          65s
```

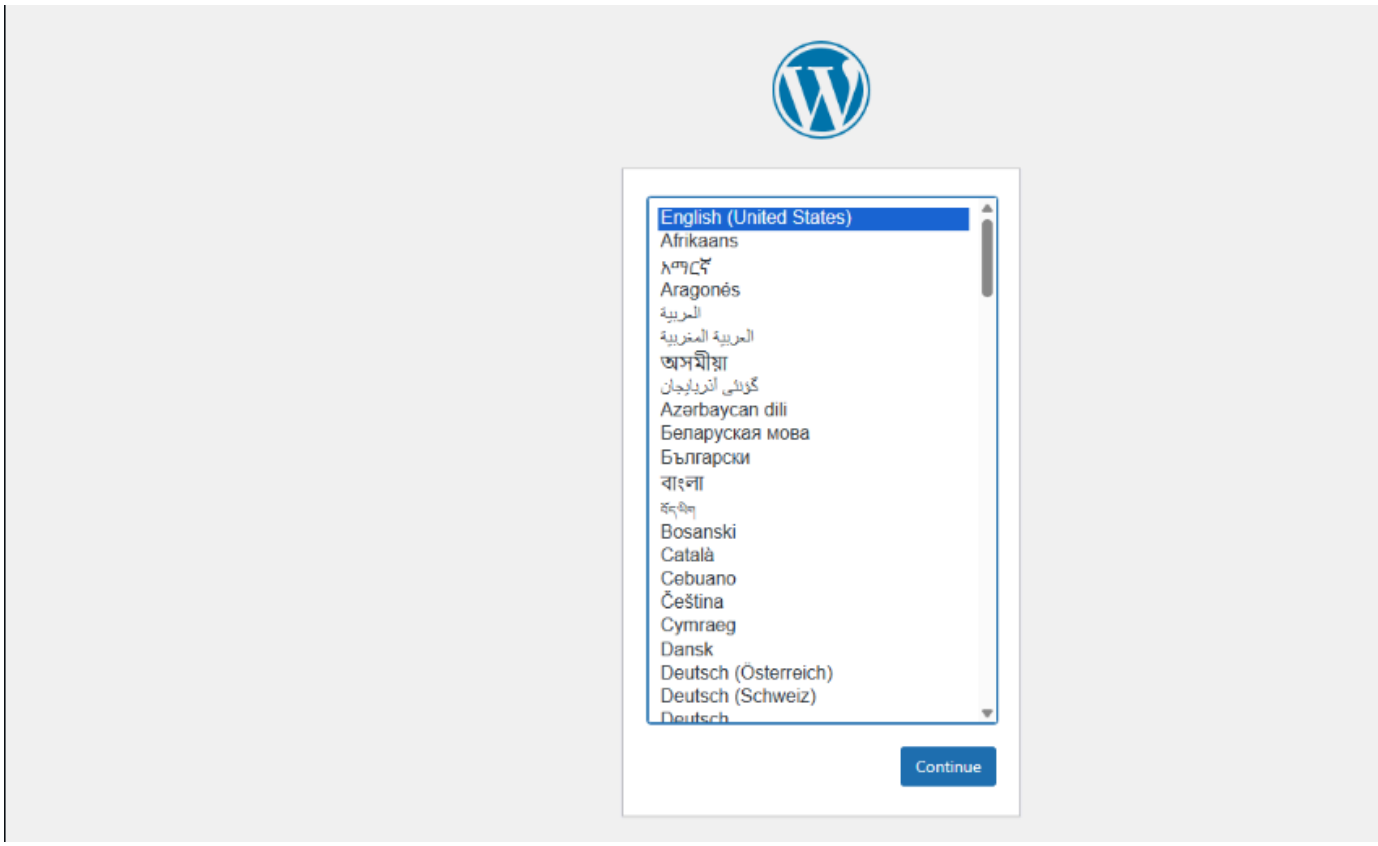
Step-6:- Check service

```
kubectl get svc
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ kubectl get svc
NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)
 AGE
kubernetes           ClusterIP   10.96.0.1       <none>        443/TCP
 2m11s
wordpress-service    NodePort    10.109.20.189   <none>        80:30007/TCP
 26s
```

Step-7:- Verify On Browser

```
http://localhost:8080/
```



Step-8:- Scale Deployment

```
kubectl scale deployment wordpress --replicas=4
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ kubectl scale deployment wo
rdpress --replicas=4
deployment.apps/wordpress scaled
```

Step-9:- Get Pods

```
kubectl get pods
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ kubectl get pods
NAME                         READY   STATUS              RESTARTS   AGE
wordpress-6698dd7d66-4gn62   0/1     ContainerCreating   0          6s
wordpress-6698dd7d66-6t6cp   0/1     ContainerCreating   0          92s
wordpress-6698dd7d66-vfw7d   0/1     ContainerCreating   0          6s
wordpress-6698dd7d66-x5bh9   0/1     ContainerCreating   0          92s
```

Step-10:- Delete Pods to check Self-healing

```
kubectl delete pod <pod-name>
kubectl get pods
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ kubectl delete pod wordpres
s-6698dd7d66-4gn62
```

Insights

When to Use Which Tool

Tool	Best for
k3d	Quick learning on your laptop
Minikube	Single-node cluster testing
kubeadm	Real, production-style cluster

Summary of Commands (Cheat Sheet)

Goal	Command
Apply a YAML file	`kubectl apply -f file.yaml`
See all pods	`kubectl get pods`
See all services	`kubectl get svc`

Goal	Command
Scale a deployment	`kubectl scale deployment <name> --replicas=N`
Delete a pod	`kubectl delete pod <pod-name>`
See all nodes	`kubectl get nodes`